# Auth-Server vs authentication-server: What Changed and Why It Matters

A side-by-side comparison of the **old** central authentication server ( `ESSI-Ph-2/authentication-server` ) and the **new** one ( `Shah-Secret-Project/Auth-Server` ).

Every statement about the old version was checked in its code, and file paths are given where it helps. The old version did several things right, and section 1 says which, so this is a fair comparison rather than a sales sheet.

## At a glance

| Area | Old: authentication-server | New: Auth-Server |
|---|---|---|
| **How a user signs in to an app** | Each app shows its own login form and sends the password to the auth server ( `POST /login/` with `app_key` ) | One hosted login page. Apps redirect to it (OAuth 2.1 Authorization Code + PKCE) and never see a password |
| **Single sign-on** | None: every app is a separate login | Yes: sign in once, every app opens without asking again |
| **Token signing** | HS256 with a **shared secret**, default value `SHAH-SUPREME-SECRET-KEY` | ES256 key pairs. Only the auth server holds private keys; apps get public keys (JWKS) |
| **What a downstream server needs to verify a token** | The **same secret** that signs tokens, so any downstream server can also forge them | Only public keys, fetched automatically. A downstream server can verify but never forge |
| **Signing key rotation** | None: one secret, forever | Automatic every 30 days. The next key is published before use, and a leaked key can be revoked in one click |
| **Downstream integration** | Copy a "handover" folder of files into each server. Its own guide lists four known defects to fix first | A versioned Python **SDK** ( `pip install -e sdk` ). Protecting a route is `Depends(auth.require_perm("x"))` |
| **Downstream server credentials** | Service clients exist, but setup is manual and the downstream server also needs the shared JWT secret | Create a client in the console, and the secret is shown once. The SDK fetches, caches and renews service tokens itself |
| **Revocation events** | Redis pub/sub. Not stored, not signed, no replay: anything sent while a server is disconnected is lost. A 10-minute user snapshot covers the gaps | Transactional outbox: stored, **signed**, numbered without gaps, replayable, and acknowledged by each server. A snapshot is used only when a server was away past retention |
| **Downstream infrastructure** | Each downstream server needs its **own Redis** for the cache | No Redis needed downstream. The SDK keeps state in memory, plus an optional file |
| **Token versioning** | One version per user for **all apps**: any change invalidates the user's tokens everywhere | One version per user **per app**: a role change in NVR leaves ANPR untouched |
| **Two-step sign-in** | TOTP only, with weaknesses (section 3.4) | TOTP with confirmation, replay protection and attempt limits, **plus passkeys and security keys** (WebAuthn) |

| Area | Old: authentication-server | New: Auth-Server |
|---|---|---|
| **Passkeys / security keys** | None | Windows Hello, Touch ID, YubiKey, phone passkeys. Several per user, each named |
| **Browser and desktop app registration** | None: no OAuth clients and no redirect URIs | Public clients for web and desktop apps, with exact redirect URIs, PKCE and no secrets |
| **Where the console keeps tokens** | `localStorage`, where any script on the page can read them | Nowhere: the console uses an HttpOnly cookie plus a CSRF token (backend-for-frontend) |
| **Sessions** | No session concept: you can't see or end one login | Every sign-in is a session with device, IP and method. Users and admins can end any of them |
| **Permissions in the token** | A list of permission code strings | A compact bit mask (up to 1,024 per app). Bits are never reused, so an old token can't gain a new meaning |
| **Role identity for downstream servers** | Roles by name only | Each role has a stable id that survives renames |
| **Listing an app's users** | `GET /get-users-by-app/{app_key}` is **public**: anyone, with no token, gets every user with their roles and permissions | `GET /v1/apps/{key}/members` needs a service token with `users.read`, returns only the caller's own app, and pages its results |
| **Default administrator** | Auto-created `admin` / `Admin@123` | No default account. `create-admin` asks for a password and works only once |
| **Account discovery** | Different messages for "wrong password" and "no access", so valid usernames can be found | One message for every failure, and equal timing for unknown usernames |
| **Brute-force protection** | No login or 2FA rate limits | Failures counted per IP, per account and IP, and per account, with `Retry-After`. Fails closed if Redis is down |
| **Error responses** | `{"detail": str(e)}`: internal errors are sent to the caller | A generic `internal_error`. Details stay in the server log |
| **Audit trail** | An `AuditLog` table, written for a few actions (mainly login) | Every change, in the same transaction, insert-only (a database trigger blocks edits and deletes), with a filterable console page |
| **Unsafe settings** | Accepted, including default secrets in `config.py` | The server refuses to start with a weak `MASTER_KEY`, wildcard CORS, or (in production) a default database password or non-https issuer |
| **Secrets at rest** | TOTP secrets stored as plain text | TOTP secrets, signing keys and console tokens encrypted (AES-GCM) |
| **Standards** | Custom endpoints; token exchange loosely modelled on RFC 8693 | OAuth 2.1, OpenID Connect (discovery, ID tokens, userinfo), PKCE, JWKS, RFC 7009, RFC 8693, RFC 9207 |
| **Admin console** | Apps, roles, permissions, users, user mapping, downstream services | Dashboard, applications (permissions, roles, members, clients), users (access, sessions, MFA, profile), clients, signing keys, downstream sync health, audit log, My account |
| **Self-service for users** | Enable 2FA only | My account: access, password, sessions, authenticator app, security keys |

| Area | Old: authentication-server | New: Auth-Server |
|---|---|---|
| **Tests** | 29 (policy and service auth) | 319 server, 24 SDK, 36 client unit, plus browser end-to-end tests against a real server |
| **Documentation** | Integration guides, plus a handover folder with known defects | Architecture, API reference, integration guide, features list, SDK README, plans and handoffs |

# 1. What the old version already got right

The new version kept these ideas and built on them:

- **Service clients** with secrets hashed using Argon2 ( `app/server_auth` ).
- **Refresh tokens** stored only as SHA-256 hashes, and rotated.
- **A token version** ( `ver` ) in the access token, so tokens could be made stale.
- **An SSE event stream** to downstream servers.
- **A central access policy** with one guard ( `security/api_policy.py` , `enforcement.py` ).
- **Token exchange** between services.

The new version keeps the ideas and fixes how each one was done, as the rest of this document explains.

# 2. Architecture

## 2.1 Old: shared secret, per-app login, Redis everywhere

```
App's own login form ──password──> /login/ (app_key) ──> HS256 token (shared secret)
                                                                  |
Downstream server ── needs the SAME secret to verify ─────────────┘
       |
       ├── its own Redis: users cache, token versions, service token
       ├── SSE from Redis pub/sub (not stored, not signed)
       └── every 10 min: GET users snapshot
```

**Problems with this design:**

1. **Whoever can verify can forge.** HS256 uses one key for signing and checking. Every downstream server holds `AUTH_SECRET_KEY` , so a bug, a leaked `.env` or one curious developer on any of them can mint an admin token for every app. The integration guide even prints the default value ( `AUTH_SECRET_KEY=SHAH-SUPREME-SECRET-KEY` ).
2. **No key rotation.** Changing the secret means editing and restarting every downstream server at the same moment, so in practice it never changes.
3. **Passwords pass through every app.** Each app collects the password in its own form. One badly written app (logging request bodies, a compromised frontend) exposes users' central passwords.

4. **Events can be lost.** Redis pub/sub is fire-and-forget. A downstream server that is restarting or briefly disconnected misses "user disabled", and finds out only at the next 10-minute snapshot.
5. **Redis on every downstream server.** It's one more thing to install, secure and monitor per app.

### 2.2 New: asymmetric keys, hosted login, signed outbox

```
 App ──redirect──> hosted login page ──> code + PKCE ──> ES256 tokens (private key stays on the auth
server)
                                                                  │
 Downstream server ── public keys only (JWKS) ────────────────────┘
        │
        └── SDK: verifies locally, follows a stored, signed, numbered event stream,
                 ACKs each event, replays after a disconnect, snapshots only when far behind
```

**What this gives you:**

1. **Only the auth server can sign.** Downstream servers hold public keys, which can't create tokens.
2. **Keys rotate on their own,** and the next key is published before it signs anything, so no downstream server is ever caught out.
3. **Apps never see passwords.** They receive a short-lived code, then tokens made only for them.
4. **Events can't be lost.** The change and its event are saved in one database transaction. Each app's events are numbered without gaps, so a server knows exactly where it stopped, asks for the rest, and confirms each one. The console shows how far behind each server is.
5. **Nothing extra downstream:** no Redis, no copied files.

---

## 3. Feature by feature

### 3.1 Sign-in, single sign-on and redirect login

**Old:** `POST /login/` takes `username`, `password` and `app_key`. Each app builds its own form and calls it. There's no shared session, so a user signing in to NVR and then ANPR types their password twice. There's no redirect flow, so no way to "Sign in with the Auth Server".

**New:**

- **One hosted login page** ( `/login` ) for every app. An app redirects there with `/oauth/authorize?client_id=...&code_challenge=...`. After sign-in, the user comes back with a one-time code that the app exchanges for tokens.
- **Single sign-on:** once signed in, opening another app goes straight through.
- **Silent sign-in** ( `prompt=none` ), **forced re-login** ( `prompt=login` , `max_age` ).
- **Safe return addresses:** the login page only returns to registered redirect URIs, so it can't be abused as an open redirect.

**Why it matters:** users have one password and one place to type it; apps are simpler and can't leak it.

### 3.2 Registering web and desktop apps (OAuth clients)

**Old:** there is no concept of a browser or desktop client. Every app talks to `/login/` directly.

**New:** in the console, **Clients** → **New client**:

- **Browser app** (public client): web apps and desktop apps. Exact redirect URIs, PKCE, no secret, because anything shipped to a browser or installer can be read.
- **Server** (confidential client): downstream backends. The secret is shown once and stored only as a hash. It can be rotated, and disabling it ends its tokens everywhere.

**Why it matters:** each app is a named, controllable identity. You can see it, limit its scopes, rotate its secret or switch it off, without touching other apps.

### 3.3 Downstream integration: the SDK vs copied files

**Old:** the `handover/` folder is copied into each downstream server. Its own `INTEGRATION.md` starts with "four things will stop it from running":

1. a bad import line to delete
2. a missing `logger_config.py` to write yourself
3. two more defects

Each server then needs Redis, two background workers wired into its lifespan, and the shared JWT secret. Every copy can drift, and a fix must be pasted into every server again.

**New:** one package, versioned (now **0.3.0**), installed with `pip install -e sdk`:

```
auth = AuthSDK(issuer=..., app_key="SNVR", client_id="snvr-a", client_secret=...)

@app.get("/cameras")
async def cameras(user: Principal = Depends(auth.require_perm("camera.view"))):
    ...
```

The SDK does the rest by itself:

- **Tokens:** checks them locally and fetches the public keys.
- **Service token:** gets it and renews it before it expires.
- **Events:** follows the stream, acknowledges each event, replays after a disconnect, and snapshots when far behind.
- **Permission names:** keeps the permission map current, so new permissions work with no restart.
- **Outages:** keeps working through them, with an optional state file for restarts.
- **Reconnects:** backs off with jitter when the auth server is down.
- **Users and roles:** lists the app's members and roles, with an `on_change` hook to refresh caches.

It has 24 tests of its own, and a live run against the real server.

**Why it matters:** this was the biggest source of integration problems. A new downstream server is now a few lines of code, not a copy-and-repair job.

### 3.4 Two-step sign-in (TOTP)

**Old** ( `router_auth.py` ):

| Weakness | Effect |
|---|---|
| `enable-2fa` turns 2FA on and saves the secret immediately, before the user proves they scanned the QR code | A user who closes the dialog is locked out of their own account |
| `totp.verify(...)` with no "already used" check | A code can be replayed within its 30 seconds |
| `verify-2fa` has no attempt limit; the `two_fa_token` stays valid for 5 minutes | Someone with the password can try codes freely |
| The secret is stored as plain text in `users.secret_2fa` | A database leak exposes every authenticator |
| The QR issuer is hard-coded to `J&K-Surveillance` | The wrong name shows in authenticator apps |
| No way to remove 2FA, and no admin reset | A lost phone needs a manual database edit |

**New:**

- **Confirmed before it counts:** a new authenticator protects the account only after its first correct code.
- **Password to set up, code to remove.**
- **Each code works once** (conditional update on the last used time step).
- **Five wrong codes destroy the challenge,** with a per-user limit across challenges.
- **Secrets encrypted** with AES-GCM.
- **Admin reset** for a lost phone, which also ends every session.
- **QR code** generated in the browser, with the right issuer.

### 3.5 Passkeys and security keys (WebAuthn)

**Old:** none.

**New:** Windows Hello, Touch ID, YubiKey and phone passkeys as a second step.

- **Keys:** several per user, each named, with a password needed to add or remove one.
- **Challenges** work once.
- **Cloned-key detection** through the signature counter.

Sign-in records how the user proved who they are ( `amr: pwd, hwk` ), which is shown in sessions and the audit log.

### 3.6 Tokens and revocation

| | Old | New |
|---|---|---|
| Algorithm | HS256 (shared secret) | ES256 (key pair) |
| Audience | `app` claim | `aud` claim, strictly checked: an NVR token is refused by ANPR |

| | Old | New |
|---|---|---|
| Permissions | List of strings | Bit mask, compact and exact in JavaScript, with bits never reused |
| Version | Per user, all apps | Per user per app |
| Session id | None | `sid` : one login can be ended on its own |
| Refresh | Rotated, 30 days | Rotated, ends with the session (14 days), **theft detection** (reuse ends the session), and a 10-second **two-tab grace window** so tabs don't log each other out |
| Revocation reach | Pub/sub plus a 10-minute snapshot | Signed, stored events, usually within a second; the 10-minute token lifetime is the guaranteed upper limit |

**The per-app version matters in practice.** In the old version, changing a user's role in one app (or re-publishing their version at login, which `user_tokken_data` does on every sign-in) affects their tokens for every app. In the new version, only that app's tokens go stale.

### 3.7 Users, roles and permissions management

**Old:** apps, roles, permissions, user assignment and a user-mapping page.

**New**, everything from the old version plus:

- **The key-cut strip:** each permission bit is drawn as a key tooth, so you can see at a glance exactly what a user holds in an app.
- **Retired permissions:** a deleted permission keeps its bit forever, and recreating its code revives it. This avoids "the old token now means something else".
- **Protected objects:** the AUTH app, its Admin role and the `*` permission can't be broken by accident. **At least one administrator always remains**, with the check locked against two admins removing each other at the same moment.
- **Users:** search, filter by status and app, an MFA badge, sessions, MFA factors, set password, disable/enable (which ends sessions and tells every app at once).
- **Members and roles for downstream servers:** `users.read` , stable role ids, and change events.

### 3.8 Sessions

**Old:** none. A refresh token exists, but you can't see who is signed in or end a login.

**New:**

- **Session list:** every sign-in has a session with device, IP, sign-in method and expiry.
- **Users** see and end their own sessions.
- **Admins** see and end anyone's sessions.
- **Automatic endings:** changing your password ends your other sessions. A password reset, MFA reset or disable ends all of them. A stolen refresh token ends its session.

## 3.9 Security hardening

| Risk | Old | New |
|---|---|---|
| Default admin password | `admin` / `Admin@123`, auto-created | None; `create-admin` asks for one, once |
| Default secrets | `SHAH-SUPREME-SECRET-KEY` in code | `MASTER_KEY` required, checked for length and randomness; the server won't start otherwise |
| Public user list | `GET /get-users-by-app/{app_key}` open to anyone | Service token, `users.read`, own app only |
| Account discovery | Different errors reveal valid usernames | One message, equal timing |
| Brute force | No limits | Rate limits on login, MFA and password confirmations |
| Internal errors | `str(e)` returned to the caller | Generic error; details logged only |
| Browser token storage | `localStorage` | No tokens in the browser (HttpOnly cookie + CSRF) |
| Login CSRF | — | State binding cookie on console sign-in |
| Security headers | — | `nosniff`, `DENY` framing, `no-referrer`, HSTS in production |
| Open redirect | — | Exact redirect URIs, and no redirect before the URI is proven |

## 3.10 Audit trail

**Old:** an `AuditLog` table, written by `log_user_action` from a handful of places (mainly login).

**New:**

- **Every change** is recorded with who, what, target, app, details and IP: applications, permissions, roles, users, sessions, MFA, clients, keys, token exchange and token theft.
- **Written in the same transaction** as the change, so the log can't miss anything.
- **Insert-only:** a database trigger rejects UPDATE, DELETE and TRUNCATE.
- **A console page** with filters kept in the URL.

## 3.11 Operations

| | Old | New |
|---|---|---|
| Downstream health | A downstream services page | Per-instance sync status: last event applied, lag, last seen, In sync / Behind / Offline, auto-refresh |
| Signing keys | One secret in `.env` | Key lifecycle page: next, active, retired, revoked; rotate now; revoke a leak |
| Background jobs | — | Cleanup, outbox pruning, key rotation and downstream reporting, safe on several replicas (Postgres advisory locks) |
| Health checks | — | `/health/live` and `/health/ready` |

| | Old | New |
|---|---|---|
| Failure behaviour | Downstream depends on Redis and the shared secret | Documented per failure: auth server down, Redis down, Postgres down, key leaked |

### 3.12 The web console

**Old:** React with Redux; tokens kept in `localStorage`.

**New:** React 19, TypeScript, TanStack Router and Query.

- **Design:** a locked design, light and dark themes, phone layout (tables become cards).
- **Accessibility:** keyboard and screen-reader support.
- **Speed:** a 128 KB login page.
- **Typed API client:** generated from the server's schema, so an API change that breaks the console fails the build.

### 3.13 Quality and documentation

| | Old | New |
|---|---|---|
| Server tests | 29 | 319, against a real Postgres |
| SDK tests | — (no SDK) | 24, plus a live run |
| Client tests | — | 36 unit and component tests, plus browser end-to-end tests |
| Access policy check | `python -m security.audit` | `python -m app.security.policy_audit`; fails if any route has no explicit rule |
| Docs | Integration guides; handover with known defects | Architecture, API, integration, features, SDK README, plans, handoffs, this comparison |

## 4. Moving a downstream server from old to new

| Old step | New step |
|---|---|
| Copy `handover/`, fix its four defects | `pip install -e path/to/Auth-Server/sdk` |
| Set `AUTH_SECRET_KEY` (the shared secret) | Remove it. Set `issuer`, `app_key`, `client_id`, `client_secret` |
| Run Redis for the user cache | Remove it. Optionally set `state_path` |
| Start `service_token_worker` and `user_token_version` tasks | `await auth.start()` / `await auth.stop()` in the lifespan |
| Read users and versions from Redis in each guard | `Depends(auth.current_user)`, `require_perm(...)`, `require_role(...)` |
| Call the public `get-users-by-app` | `await auth.list_all_members()` with `users.read` |
| Build your own login form | Redirect to the hosted login page (Authorization Code + PKCE) |

The full guide is in `auth-server/docs/INTEGRATION.md`.

## 5. Summary

The old server proved the idea of one auth service for many apps. The new server makes it safe and easy to live with:

- **Safe:** only the auth server can sign, keys rotate, passwords never pass through apps, nothing is public that shouldn't be, and every change is on record.
- **Reliable:** revocations are stored, signed, ordered and confirmed, so no "missed event" windows.
- **Easy to integrate:** one SDK and a few lines per server, instead of copied files, Redis and a shared secret.
- **Modern sign-in:** single sign-on, a hosted login page, passkeys, a properly built TOTP flow, and registered web and desktop apps.
- **Manageable:** fine-grained permissions, stable role ids, sessions, a downstream sync view, key management and a full audit log.